Stacked Arhat

I. Gameplay Introduction

```
    The purpose of this experiment is the opposite of the previous lesson 'Nature Porter'. It is to grab building blocks from different surrounding areas in the order of yellow, red, green, and blue, and stack them in the middle gray area.

    The arrangement of building blocks is shown in the figure below:
```

After executing the code, the robotic arm will stack the building blocks. The final effect is shown in the figure below:

II. Start Docker

After entering the main board desktop, open a terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then enter the following command in the Docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address followed by :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

III. Code Content

Code path: /root/Dofbot/3.ctrl_Arm/11.heap_up.ipynb

```
#!/usr/bin/env python3
#coding=utf-8
import time
from Arm_Lib import Arm_Device
# Create robotic arm object
Arm = Arm_Device()
time.sleep(.1)
```

```python
# Define block clamping function, enable=1: clamp, =0: release
def arm_clamp_block(enable):
     if enable == 0:
         Arm.Arm_serial_servo_write(6, 60, 400)
     else:
         Arm.Arm_serial_servo_write(6, 130, 400)
         time.sleep(.5)

# Define robotic arm movement function, simultaneously control servos 1-5
movement, p=[S1,S2,S3,S4,S5]
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        elif id == 1 :
            Arm.Arm_serial_servo_write(id, p[i], int(3*s_time/4))
        else:
            Arm.Arm_serial_servo_write(id, p[i], int(s_time))
        time.sleep(.01)
    time.sleep(s_time/1000)
```

```python
# Define variable parameters for different positions
p_mould = [90, 130, 0, 0, 90]
p_top = [90, 80, 50, 50, 270]
p_layer_4 = [90, 76, 40, 17, 270]
p_layer_3 = [90, 65, 44, 17, 270]
p_layer_2 = [90, 65, 25, 36, 270]
p_layer_1 = [90, 48, 35, 30, 270]
p_Yellow = [65, 22, 64, 56, 270]
p_Red = [118, 19, 66, 56, 270]
p_Green = [136, 66, 20, 29, 270]
p_Blue = [44, 66, 20, 28, 270]
```

```python
# Move the robotic arm to a preparation position for grabbing
arm_clamp_block(0)
arm_move(p_mould, 1000)
time.sleep(1)
```

```python
# Grab the block from the yellow area and stack it at the bottom layer position
in the middle.
arm_move(p_top, 1000)
arm_move(p_Yellow, 1000)
arm_clamp_block(1)
arm_move(p_top, 1000)
arm_move(p_layer_1, 1000)
arm_clamp_block(0)
time.sleep(.1)
arm_move(p_mould, 1100)

# time.sleep(1)
```

```
# Grab the block from the red area and stack it at the second layer position in
the middle.
arm_move(p_top, 1000)
arm_move(p_Red, 1000)
arm_clamp_block(1)
arm_move(p_top, 1000)
arm_move(p_layer_2, 1000)
arm_clamp_block(0)
time.sleep(.1)
arm_move(p_mould, 1100)
# time.sleep(1)
```

```
# Grab the block from the green area and stack it at the third layer position in
the middle.
arm_move(p_top, 1000)
arm_move(p_Green, 1000)
arm_clamp_block(1)
arm_move(p_top, 1000)
arm_move(p_layer_3, 1000)
arm_clamp_block(0)
time.sleep(.1)
arm_move(p_mould, 1100)

# time.sleep(1)
```

```
# Grab the block from the blue area and stack it at the fourth layer position in
the middle.
arm_move(p_top, 1000)
arm_move(p_Blue, 1000)
arm_clamp_block(1)
arm_move(p_top, 1000)
arm_move(p_layer_4, 1000)
arm_clamp_block(0)
time.sleep(.1)
arm_move(p_mould, 1100)

# time.sleep(1)
```

```
del Arm # Release the Arm object
```